Visually-Grounded Library of Behaviors for Generalizing Manipulation Across Objects and Views

Anonymous Author(s)
Affiliation
Address
email

Abstract: We propose a visually-grounded library of behaviors approach for manipulating diverse objects across a wide range of initial and goal configurations and camera placements. Our key innovation is to disentangle the standard image-to-action or image-to-state-to-action process into two separate modules that use different types of perceptual input: (1) a behavior selector which conditions on the *invariant* object properties to select the behaviors that can successfully perform the desired tasks on the object in hand, and (2) a library of behaviors each of which conditions on the *variable* object properties to predict the manipulation actions over time. To be able to handle varying camera views, the selector operates on semantically-rich and affordance-aware view-invariant 3D object feature representations extracted from images using geometry-aware 2D-to-3D neural networks. Our model encodes affordance information in the object feature representations by predicting outcomes of applying manipulation behaviors to objects, in a self-supervised manner. The library consists of different types of behaviors that condition on more succinct object representations which capture the dynamics of the objects (such as 3D centroids or poses of the objects) and maps them to robot actions, given a goal. We test our framework using simulated grasping and pushing of a variety of objects between a variety of initial and goal configurations, with a range of camera viewpoints. Our approach outperforms image-to-action mappings or object-to-action mappings that do not factorize changing and invariant visual object properties, as well as position-to-action mappings that do not consider any contextual visual features.

1 Introduction

Current visuomotor policies for robots typically do not generalize well across objects and views [1]. This is particularly true for image-based policies learned end-to-end [2, 3, 4]. Auxiliary tasks, such as autoencoding [5], object and gripper detection [6], Imagenet classification [7], and depth and edge detection [8] have been used to help visual feature representations generalize better with less interactions. Lack of generalization of end-to-end image-to-action mapping is attributed to:

1. Lack of state abstraction. It is often useful to find a state abstraction, a minimal subset of environment features useful for performing a task. Abstractions facilitate transfer by explicitly ignoring parts of the environment that are irrelevant to a particular skill. Only a few object properties or sensory signals will usually be relevant for a given task [9]. For example, when opening a bottle, the size of the cap is a relevant feature, but the color of the bottle itself is irrelevant. Images have rich contextual information that may be irrelevant to the task and policies that attempts to learn what features matter automatically often require enormous amounts of training examples.

2. Entanglement of static and dynamic object properties in images. Object visual properties are categorized into dynamic ones that change over the course of an episode, such as 3D object locations, and static ones that vary across episodes and versions of the task, such as object shape, appearance, size, mass, and friction. The entanglement of static and dynamic properties, also termed contextual and state properties in previous work [9], makes it hard to learn concise policy models.

3. Occlusions, foreshortenings, and other artifacts caused by camera projection in images. As a result, appearance and location of the object and the gripper and their relative arrangement, which may be constant in 3D, varies with the viewpoint in 2D images.

Submitted to the 4th Conference on Robot Learning (CoRL 2020). Do not distribute.



Figure 1: The goal of this paper is to learn manipulation behaviors that can generalize across different objects, configurations, and camera views. Consider the task of clearing a table. "2D" grasps where the hand points down and approaches the object vertically do not work for plates and bowls. For the coffee cup and soda can, many grasps shown are not robust. A grasp planner that did not understand which surfaces are dirty or if a can is full or empty would grasp objects in undesired ways. Natural objects like the onion are often more complex than man-made objects. In this case, many grasps that looked good failed because the onion skin slid off the onion, or the stem tore off, or the onion rolled out of the grasp due to low torsional friction. Our approach learns which grasping behavior to apply in each object configuration.

Developing vision-based policies that can work across objects and views remains an open problem, and it is unclear how to integrate visual context to inform the policy mapping about the static object properties, without making the mapping harder to learn. There is different functionality of the contextual features and task-based features, whose factorization may help learn more concise and generalizable sensory-to-action mappings. Lack of state abstraction, entanglement of context and state, and projection artifacts make mapping of images to actions hard to learn. This has led classical robotics approaches to advocate for known 3D object models, simplifying the state inference problem [10].

In place of a single image-to-action mapping, we propose a Visually-grounded library of BEhaviors (V-BEs) as a manipulation policy representation that handles variability of the manipulation task caused by varying objects, varying initial and goal configurations, and varying viewpoints (Figure 1). In V-BEs, a gating network selects plausible behaviors to execute, given an object of interest and a goal, from a library of possible behaviors, shown in Figures 2 and 3.

We define a behavior to be a mapping from (a subset of) the dynamic part of the environment's state—e.g., the objects' 3D centroids—to gripper actions. Behaviors can be redundant, have different input state representations, and be learned with different algorithms [11, 12], or be manually engineered controllers [13]. We define the selector to be a mapping from a viewpoint-invariant 3D object feature representation to behaviors that can successfully manipulate the object. Such an object representation is learned via a combination of self-supervised view-prediction and object interaction prediction. By training the model for a view prediction task using multi-view images, we bias the representation to be viewpoint-invariant. By training the model with labels obtained from interactions which reflect the affordance of an object (i.e., how objects change after applying a behavior), the representation encodes affordance information. The selector operates as a retrieval method, and can accommodate a growing library of behaviors.

We test our representation using simulated grasping and pushing with a large pool of diverse objects. We show that the proposed representation can learn to grasp and push diverse objects in various ways and to various end-goals, and can handle camera viewpoint variability. It far outperforms existing single-policy image-to-action mapping [4], or object-to-action mapping that uses only the fast changing 3D object locations [14]. Our code will be made publicly available upon publication.

2 Related Work

Libraries of Behaviors Libraries of behaviors have been considered in many previous works, and library elements have been called action, motion, or motor primitives, parameterized policies, options, chunks, macros, subroutines, behavioral units, and skills, as well as many other terms. Most previous work does not use perception, but rather assumes the environment state is known, or has a different approach to perception than ours. Work of Neumann et al. on modular policies [15] defines

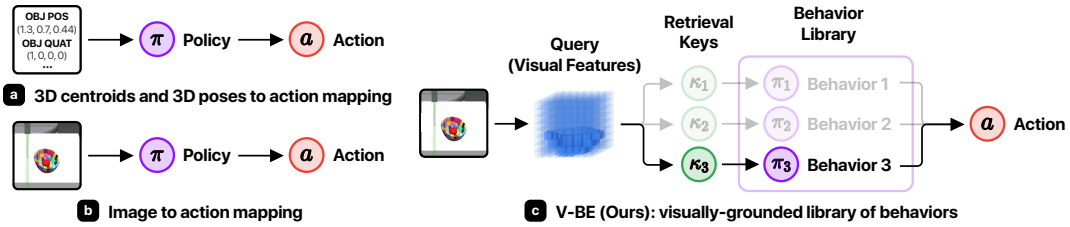


Figure 2: Comparison of our model versus previous approaches. The proposed V-BEs decomposes a policy into a behavior selection module and a library of behaviors to select from. This decomposition enables different state representations to be used for each behavior and the selection module.

a motor primitive as a mapping from a robot's joint angles and velocities and not the locations of objects. Rather, object locations are used as context for selecting a motor primitive. This means i) motor primitives are "blind" to the object 3D location throughout the episode, and ii) object appearance is discarded which means the library of primitives cannot handle object variability. Work of de Silva [16] on parameterized policies similarly considers as the contextual vector the desired goal location of a dart, and learns a mapping from this location to policy parameters, where each policy operates only over the robots' joints and is represented as a Dynamic Motion Primitive (DMP) [17, 18, 19]. DMPs leverage the fact that many robotic movements can be decomposed into two parts—a goal configuration and a "shape" of the motion, and represent position and force trajectories well. DMPs are less suitable for physically interactive behaviors. DMP controllers can be learned from very little data, but typically only generalize well in cases in which a good solution policy broadly can be described by a single prototypical motion shape primitive. Kroemer et al. [20] parameterize DMPs also based on object features, such as the distance of an object part from the gripper, but only consider known object 3D shapes. Strudel et al. [21] use a depth frame as input to a meta-policy that selects skills to execute. However, the use of modularity over skills is for temporal sequencing: there is no mixture of behaviors per manipulation task, rather an image to action mapping is learnt with behavior cloning from demonstrations. Tool selection based on shape is similar to our shape-based behavior selection [22].

Deep Reinforcement and Imitation Learning for Object Manipulation In current work in (deep) reinforcement and imitation learning, typically one policy is learned via reinforcement learning [12, 23, 24], imitation learning [25, 26, 27], or a combination of the two [28, 29]. Deep RL algorithms are commonly trained and evaluated in a fixed environment in terms of their ability to discover a policy rather than their ability to generalizing it to previously unseen circumstances. Almost all of this literature parameterizes the learnt policies in terms of object and gripper 3D centroids and their velocities, and ignores contextual visual information [30]. These policy parameterizations are often focused on discovering a skill for a specific object, and do not generalize to other objects. Several works have suggested that image-based policies can be discovered as well but more slowly [31], or use state information (object centroids) for the critic network, while learning an image-based policy network that can operate directly from images at test time [32]. Imitating planners that distill their trajectories into image-based rich mappings benefit from the planners' low-dimensional state space [7, 33]. These methods cannot generalize across objects and environments, as also pointed out in [34]. Relational policy architectures attempt to generalize across numbers of objects and their placement in the image using self-attention layers [35].

Florence et al. [36] train manipulation policies parameterized by the 2D or 3D locations of a designated set of visual descriptors. They pretrain their visual feature space using correspondences from triangulation and augmentation of images by resizing and warping. While [36] shows their policies generalize across objects of the same object category, e.g., shoes of different shapes, our policy representation further allows generalization across object categories, and no explicit descriptor selection or optimization is necessary.

3 Visually-Grounded Library of Behaviors (V-BEs)

Our policy representation Π is a visually-grounded library of behaviors, as shown in Figure 2. V-BEs are comprised of multiple behaviors $\{\pi_i \mid i = 1, 2, \ldots, K\}$ and a behavior selector G. Each behavior π_i is either an open loop trajectory generator, or a closed-loop policy (feedback controller) that tracks objects over time. Since most behaviors are developed using abstract state representations,

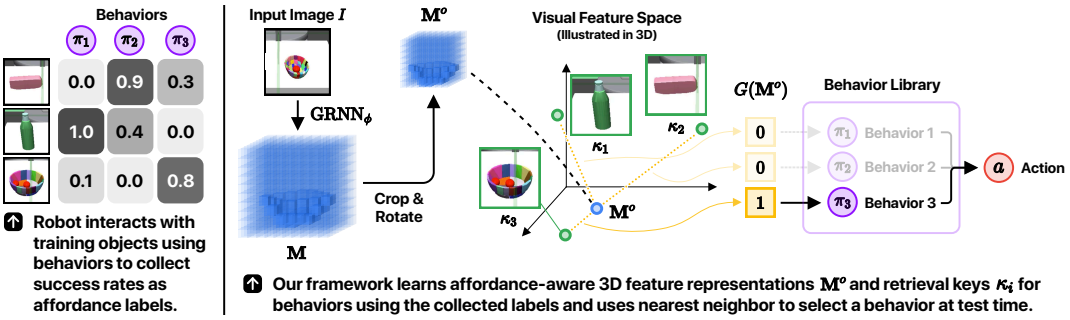


Figure 3: Overview of the proposed framework.

namely, 3D object locations and 3D poses and do not take into account object shape or appearance, they can handle only a subset of objects and their 3D orientations. To be able to handle a diverse range of objects, we need to decide which behavior to use under which circumstance. To achieve this, we propose a behavior selector which is a gating network that, given the sensory observations o_t, predicts the behavior that can best manipulate the object under consideration to its desired configuration. Our sensory input to action mapping then, at each time-step reads:

$$\Pi(a_t|o_t, g) = \sum_{i=1}^{K} G(o_t)_i \pi_i(a_t|o_t, g), \tag{1}$$

where g is the goal configuration to achieve, $o_t = \{\mathbf{I}_{\mathbf{v_t}}, \mathbf{v}_t\}$ is the sensory observation in the form of one or more RGB-D images $\mathbf{I}_\mathbf{v} \in \mathbb{R}^{\hat{H}\times\hat{W}\times 4}$ captured from arbitrary but known camera poses $\mathbf{v} \in \mathbb{R}^7$, and $G(o_t)_i \in \{0, 1\}$ denotes the selector's output for π_i at time step t. Our main contribution is the factorization of the visual properties of the object to be manipulated contained in the sensory observation o_t, so that, the behaviors $\{\pi_i \mid i = 1, 2, \ldots, K\}$ and the selector G operate on separate object visual properties. The behaviors use as input *dynamic* visual properties that change throughout the manipulation episode, such as object or part 3D positions. That is, each behavior can abstract away appearance information from the raw sensory input. Let $f^{\pi_i}(o_t)$ denote the state abstraction behavior i uses. In contrast, the selector G takes as input the raw sensory observation o_t and transforms it into a location invariant visual representation of the object $f^G(o_t)$, which captures object appearance and 3D size, which remain constant throughout the manipulation. With such a distinction on the representation used by the two modules, we can rewrite Equation (1) into:

$$\Pi(a_t|o_t, g) = \sum_{i=1}^{K} G(f^G(o_t))_i \pi_i(a_t|f^{\pi_i}(o_t), g). \tag{2}$$

One advantage of having this decomposition is that each behavior π_i can then use its own state abstraction $f^{\pi_i}(o_t)$, such as object 6D-poses, axis-aligned bounding boxes (as used in this paper), part-based 3D boxes, or object keypoints locations [37]. This framework supports integrating a wide range of existing models, representations, and behaviors as selectable behaviors. This architecture is also recursive, in that any behavior can consist of the same architecture replicated at a lower level. We consider here single object manipulation skills but our formulation can generalize to multiple objects.

In the rest of this section, we first explain the architecture and training details of the behavior selector in Section 3.1, and then explain how we acquire a library of behaviors for the two robot manipulation tasks we evaluate our framework on in Section 3.2.

3.1 Visually-Grounded Behavior Selector

The behavior selector G is a nearest neighbor classifier that uses the visual feature representation of the object to manipulate $f^G(o_t)$ as a query to retrieve the behavior that has the highest propabability to manipulate the object successfully. Each behavior π_i is associated with a learned retrieval key κ_i.

The behaviour selector should ideally be viewpoint invariant as the behavior that can successfully manipulate an object should not depend on the camera viewpoint the scene is viewed from. To achieve this, we design the selector G to operate on a 3D viewpoint-invariant object-centric feature

representation $f^G(o_t) = \mathbf{M}^o \in \mathcal{M} = \mathbb{R}^{H \times W \times D \times C}$, which is a set of 3-dimensional feature maps centered around the object of interest. The retrieval key $\kappa \in \mathcal{M}$ is also learned in the same representation space. How can we obtain similar 3D feature maps independent of the camera viewpoint, despite the fact that different parts of the object are visible in each? We build upon recent advances in 3D perception and use geometry-aware networks (GRNNs) [38] to map a single RGB-D image or a set of multi-view images to a complete 3D feature representation of the scene the image(s) depict. GRNNs learn to complete the missing information from a single view by optimizing end-to-end for view prediction using multi-view data collected by the robot [38]: an input RGB-D image is un-projected into an (incomplete) 3D feature grid, missing information is "inpainted" via a series of 3D convolutions, and the completed 3D feature grid is projected to a novel viewpoint and decoded to the corresponding RGB-D image. The view prediction loss reads:

$$\mathcal{L}_{\text{view-pred}}(\phi) = \sum_{n=1}^{N} \| P_\phi(\text{GRNN}_\phi(\mathbf{I}^n_{\mathbf{v}^n}, \mathbf{v}^n), q) - I^n_q \|_2, \tag{3}$$

where $\text{GRNN}(\mathbf{I}_\mathbf{v}, \mathbf{v})$ is a function that lifts RGB-D input images $\mathbf{I}_\mathbf{v}$ capturing from camera poses $\mathbf{v}$ to a geometry-consistent 3D feature map $\mathbf{M} = \mathbb{R}^{\bar{W} \times \bar{H} \times \bar{D} \times \bar{C}}$ with $\bar{W}, \bar{H}, \bar{D}$ denoting the spatial dimension and $\bar{C}$ denotes the feature dimension of the 3D feature map, $P(\mathbf{M}, q)$ is a projection function that projects the 3D feature map from a query viewpoint q and decodes to a target image, I^n_q is the target image to predict, and ϕ is the neural network weights of GRNN. From the scene map $\mathbf{M}$, we obtain object-centric feature representation $\mathbf{M}^o$ by cropping the scene map using a fixed-size axis-aligned box, centered around the object we wish to manipulate.

Our selector G, given a 3D object feature representation $\mathbf{M}^o$ of the object to be manipulated and a retrieval key κ_i for behavior π_i, computes the probability $\hat{G}_i$ that π_i can successfully manipulate the object to its desired goal location:

$$\hat{G}(\mathbf{M}^o)_i = \sigma(\langle \mathbf{M}^o, \kappa_i \rangle) \in [0, 1], \tag{4}$$

where $\langle \cdot, \cdot \rangle$ is the inner product operation and σ is the sigmoid function. It then selects the behavior with the highest predicted probability:

$$G(f^G(o_t))_i = G(\mathbf{M}^o)_i = \mathbb{1}\{i = \text{argmax}_{i'} \hat{G}(\mathbf{M}^o)_{i'}\}. \tag{5}$$

Affordance-aware Feature Learning Supervised by Interaction The 3D feature representations obtained through view prediction capture how objects look, but not how they can be manipulated. Our selector should operate over representations that capture information about how an object can be used, and how it will react to an agent's actions. We will use the term *affordance*, coined by Gibson in 1966 [39], to denote this information.

We learn affordance-aware 3D feature representations by applying behaviors in the library to the objects in the training set, and update the object representations and the retrieval keys $\kappa_i, i = 1 \cdots K$ for behaviors $\pi_i, i = 1 \cdots K$ according to the success rate a behavior achieves on a particular training object, as shown in Figure 3. Let $u_{ij} \in [0, 1]$ be the success rate after N_{attempt} trials of behavior π_i on object ω_j, under potentially varying goals g. Here, we treat same objects with different initial orientation as different objects since we assume that canonical poses of objects are not given. Each trial encounters the same object in a potentially different initial location, orientation and desired goal location. Our loss then reads:

$$\mathcal{L}_{\text{affordance}}(\kappa_1 \cdots \kappa_K, \phi) = \sum_{i=1}^{K} \sum_{j=1}^{|\Omega_{\text{train}}|} \sum_{n=1}^{N_{\text{attempt}}} \text{BCE}\big(\mathbb{1}\{u_{ij} \geq \delta\}, \langle R_\phi(I^{\omega_j}_n), \kappa_i \rangle\big), \tag{6}$$

where $\text{BCE}(p, l) = -p \cdot \log(\sigma(l)) + (1 - p) \cdot \log(1 - \sigma(l))$ is the standard binary cross entropy loss, δ is a hyperparameter for thresholding the success rates, $I^{\omega_j}_n$ are the initial RGB-D images drawn from tasks with object ω_j in the nth trial, $R(I)$ is the 2D-to-3D visual feature extractor that takes the cropped object-centric 3D feature representations from GRNNs, and ϕ is the GRNNs weights. The final objective for training the affordance-based visual features is

$$\underset{\kappa, \phi}{\text{minimize}}\, \mathcal{L}(\kappa, \phi) = \mathcal{L}_{\text{view-pred}}(\phi) + \lambda_{\text{affordance}} \cdot \mathcal{L}_{\text{affordance}}(\kappa, \phi),$$

where $\lambda_{\text{affordance}}$ is a hyperparameter for balancing the two losses.

3.2 Building a Library of Behaviors

The visually-grounded behavior selector selects plausible behaviors to manipulate a specific object from a library of behaviors. But how can we obtain this library in the first place? Indeed, any existing behaviors, whether engineered or learned using reinforcement learning or imitation learning, can be included in our library. In this paper, we consider two common manipulation tasks: pushing and grasping, and build appropriate behavioral libraries for each.

For pushing, the behaviors are deterministic goal conditioned policies $a_t = \pi(s_t, g), t = 1 \cdots T$ that map a state of the environment and the robot $s_t = [s_t^e, s_t^r]$ to an action at every time step. In our case, the state of the environment is the 3D object centroid $s_t^e = f(o_t, O)$ and the robot state (gripper 3D location, pose, and whether it is opened or closed). The action includes 3D motion, opening (position control), and closing (force control) of the gripper. We train multiple goal conditioned policies by clustering object category and initial poses, and training a policy for each using a combination of deterministic policy gradients [28] with goal relabelling [12] while randomizing initial and goal object 3D locations. More details regarding the training setup for pushing is located in Section 4.2.

For grasping, we design controllers $\pi(a_t|g; p^{grasp}, q^{grasp})$ which given a 3D grasping point $p^{grasp} \in \mathbb{R}^3$ relative to the center of the object and a grasping 3D angle $q^{grasp} \in \mathbb{R}^{3\times3}$, move the gripper (open loop) to the grasping 3D point location, close it, and move it to the desired goal location. Specifically, the grasping points and angles have the following parameterization:

$$\text{grasp}^p(\alpha, \gamma) = \frac{1}{2}[h, w, d] \cdot \alpha + \beta, \;\; \text{grasp}^q(\gamma, \eta) = R_y(\eta) * R_z(P_{xy}(\text{grasp}^p)) * R_x(\gamma) \quad (7)$$

where h, w, d are the height, width, and depth of the object bounding boxes, $\alpha, \beta \in \mathbb{R}^3$ are parameters for determining the grasping points, $\gamma, \eta \in [-\pi, \pi]$ are the pitch and roll rotation angles on the gripper, and $P_{xy}(Z)$ is the angle between x-axis and a vector shooting from the origin to the projected point of Z on the xy-plane. Some of the designed controllers grasp the center of objects with varying angles and others grasp faces, edges, or corners. We empirically found that these parameterized controllers are quite stable and can be shared across multiple objects.

4 Experiments

We test our model on grasping and pushing a wide variety of objects in the MuJoCo Simulator [40]. Our environment consists of a Fetch Robot equipped with a parallel-jaw gripper. The robot is positioned in front of a table of height $0.4m$. To obtain the visual observations, on each episode, we choose 3 random cameras from cameras placed at 30 nominal different views including 10 different azimuths ranging from 0° to 360° combining with 3 different elevation angles from 20°, 40°, 60°. All cameras are looking at the center of the table top, and are 0.5 meter away from that point. All images have size 128×128.

Task Descriptions In the grasping task, the agent has to grasp an object and move it to a specified target location above the table. We use 274 distinct object meshes from 6 categories in ShapeNet [41] including toy buses, toy cars, cans, bowls, plates, and bottles. We randomly split the dataset into 207 objects for training, and 67 objects for testing. After augmenting the mesh with random scaling from 0.8 to 1.5, and random rotations around the vertical (z) axis, we get a total of 800 distinct object configurations (object instance and pose), 600 for training and 200 for testing. At the beginning of each episode, an object is placed in an area of $15cm \times 8cm$ around the center of the table, and a goal is sampled uniformly around the gripper's initial position $10 \sim 30cm$ away from the gripper. With a fixed time horizon of 110 timesteps (22 sec in clock time), an episode is regarded as successful if the object centroid is within $5cm$ of the goal position at the final timestep.

In the pushing task, the agent has to push an object placed on the table to a specified target location. We use 103 objects from 12 categories in ShapeNet [41]: baskets, bowls, bottles, toy buses, cameras, cans, caps, toy cars, earphones, keyboards, knives, and mugs. We randomly split the dataset into 63 objects for training, and 40 objects for testing. During training, we obtain a total of 615 different object configurations by varying the size and initial pose of the objects. We use 415 object configurations for training and the rest for testing. The initial and the goal position of the object are both uniformly sampled to be within $15cm$ of the center of the table along both x-axis and y-axis, although we resample if that location is already in the goal area. An episode is regarded as successful if the gripper pushes the object within $5cm$ of the goal position within 50 timesteps.

Baselines We compare our method with various learning and non-learning based methods for robot object manipulation:

Single Behavior w/ Abstract 3D State (Abstract 3D) [12, 28]: a policy trained with deterministic policy gradients (DDPG) [11] and hindsight experience replay (HER) [12] which takes ground truth 3D bounding box information of the object and gripper as input and outputs the target actions. The model is a 3-layer fully-connected network, identical to the architecture used for the behaviors in our method.

Single Behavior w/ 3D Bounding Box and 2D Images (Abstract 3D + Image): a policy trained with DDPG that takes as an input both 2D RGB images and an abstract 3D state and outputs the target actions. The images are transformed into a feature vector through two 2D-convolutional layers and a fully connected layer before being concatenated with the rest of the inputs.

Single Behavior w/ 3D Feature Tensor (Contextual 3D): a policy that takes a 2D image of the environment as an input, encodes the image into a 3D feature tensor using GRNNs [38], and then produces the output actions. We found this policy representation too slow to train with RL and thus used DAGGER [42] to imitate multiple pushing and grasping expert behaviours trained each on isolated objects.

6-DOF Grasp-Net (GraspNet) [43]: a state-fo-the-art grasping method that describes the grasp as a full 6-degrees-of-freedom (DoF) pose. The system learns a grasp generator that proposes potential grasp points given a point-cloud from the depth image, and an evaluation module which evaluates the quality of the proposed grasp. The system is trained with 10M grasps generated from randomly generated boxes and cylinders, and ShapeNet models of bowls, bottles and mugs. We used the publicly released code and model provided by [43] and evaluated it in our test environments.

Visual Foresight [4]: a model-based approach that uses the current frame and the action of the agent to predict future 2D frames by "moving pixels" based on predicted 2D pixel flow fields. The method uses the frame predictive model to infer an action trajectory that brings an object pixel to the desired (2D) location in the image space.

In addition to the list of baselines shown above, we also tried training a policy that took 2D images of the scene as input and output actions using both DDPG-HER [11, 12] and DAGGER [42] with generated expert trajectories. This method was not able to learn either task, resulting in a zero success rate for both grasping and pushing. Thus, we exclude it from the comparisons.

Our experiments aim to answer the following questions: (1) Does the proposed library-based approach outperform existing methods that use monolithic perception, either using 2D images, 3D object locations, or 3D scene feature maps as input to their policies? (2) Is the proposed behavior selector that uses a 3D object feature representation a necessary choice? (3) How much do the interaction labels help over view predictive object feature representations?

4.1 Grasping

Building The Behavior Library We manually selected 30 different controllers including grasps with the gripper-hand pointed downwards and moving along a vertical axis with different yaw orientations (top-grasps) and grasps from the side with different elevation angles of the gripper, and the hand moving towards the centroid of the object (side-grasps). We provide the exact parameters used for all the controllers in the supplementary materials. We found that the plates are especially hard to grasp and often require grasps from a low elevation angle $(45°)$. All objects typically slide during side-grasps, when the hand has a horizontal movement component (a scenario not addressed in work that focuses only on top-grasps). Surprisingly, a controller, whose parameters are tuned on one plate, can work on different types of plates and even provide a more robust way to grasp a large bowl that cannot be reliably grasped with a top-grasp.

Results As shown in Table 1, *Abstract 3D* performs poorly. This is due to two main reasons: first, learning grasping behaviors with DDPG-HER [11, 12] is hard as a successful grasp is a rare event and this results in extremely sparse rewards, as also pointed out in multiple previous works [12]. To successfully learn a grasping policy, we supplied demonstration trajectories as input to the experience buffer of the policy to guide reinforcement learning of the behaviour. Secondly, the learned behavior transfers poorly to new objects. Adding a 2D image helps, but not dramatically (see *Abstract 3D + Image* in Table 1). Although 3D feature maps obtained from GRNNs are semantically rich and can handle varying viewpoints, the mapping to actions is harder to learn due to the higher dimensionality of the 3D scene map. *GraspNet*, which operates on object 3D point clouds, also

performs poorly. This approach is sensitive to the size of the objects and the point cloud coordinate system. Our approach outperforms these methods by a large margin despite the fact that it only uses very naive controllers in the behavioral library.

	Abstract 3D [12, 28]	Abstract 3D + Image	Contextual 3D	GraspNet [43]	V-BEs (Ours)
Success Rate	0.30	0.35	0.20	0.23	**0.78**

Table 1: Success rates for grasping unseen objects and moving them to a target location.

4.2 Pushing

Building The Behavior Library We learn 25 behaviors for the pushing task. Among these 25 behaviors, one is trained from the whole set of objects, while the others are trained on disjoint subsets of object configurations. For eight object categories that are mostly symmetric, we train one behavior for each category and initialize the objects in their canonical poses; for buses, cars, keyboards, and knives, we train 4 policies for each object category to handle initial object poses with rotations of $0°$, $45°$, $90°$, or $135°$ along the z-axis from the canonical pose of the object, respectively. We include more training details for our method and the baselines in the supplementary materials.

Results As shown in Table 2, our method outperforms all the baselines. *Abstract 3D* performs well, but since it does not use any visual information, its performance saturates at around 0.8. The *Abstract 3D + Image* baseline performs much worse than the *Abstract 3D* baseline. We attribute this failure to the failure of the learned policy to extract and represent useful information from the image input. The *Visual Foresight* baseline completely fails on the task. The *Visual Foresight* baseline cannot predict the future accurately for more than a few timesteps, and as a result, the agent rarely even interacts with the object during the receding horizon (MPC) controller execution. Among all methods, V-BEs (ours) can generalize most robustly to the test objects due to the behavior selector's capability to reason about the objects' visual appearances and affordances.

	Abstract 3D [12, 28]	Abstract 3D + Image	Contextual 3D	Visual Foresight [4]	V-BEs (Ours)
Success Rate	0.83	0.57	0.10	0.00	**0.88**

Table 2: Success rates in pushing unseen objects.

4.3 Ablation Study: Behavior Selector Using Different Visual Representations

Next, we show the importance of learning visual affordance features with a 3D visual feature representation and the importance of self-supervised training with interaction labels. We compare our method with two baselines: (a) a model with a selector that operates over 3D visual feature representation learned only with the view and occupancy prediction loss, as suggested in [38][44], without finetuning with interaction labels, and (b) learning the visual affordance features using 2D visual features extracted from 2D CNNs. Our method significantly outperforms these two baselines, which shows the importance of fine-tuning on the execution labels as well as the use of 3D feature representations for behaviour selection.

	Proposed Method (3D Features)	Proposed Method w/o Fine Tuning on Interatcion Labels [38]	2D Features
Grasping	**0.78**	0.31	0.46
Pushing	**0.88**	0.46	0.81

Table 3: Success rates on grasping unseen objects.

5 Conclusion

We have presented a visually-grounded behavior library for manipulation that can generalize manipulation skills across objects and views. Our key insight is to use dynamic object properties that change during manipulation to parameterize the behaviors, and use viewpoint-invariant object appearance representations as context to select the appropriate behavior to use for each object and pose. We show the proposed policy representation outperforms alternatives that use entangled visual representation and do not factorize in this way state and visual context features. Extending the proposed representation towards model-based control by sequencing the behaviors over time to discover longer duration policies and learning diverse state representations for multi-object or part based interactions are clear avenues for future work.

References

[1] S. Dasari, F. Ebert, S. Tian, S. Nair, B. Bucher, K. Schmeckpeper, S. Singh, S. Levine, and C. Finn. Robonet: Large-scale multi-robot learning, 2019.

[2] P. Agrawal, A. Nair, P. Abbeel, J. Malik, and S. Levine. Learning to poke by poking: Experiential learning of intuitive physics. *CoRR*, abs/1606.07419, 2016. URL http://arxiv.org/abs/1606.07419.

[3] C. Finn, T. Yu, T. Zhang, P. Abbeel, and S. Levine. One-shot visual imitation learning via meta-learning. *CoRR*, abs/1709.04905, 2017. URL http://arxiv.org/abs/1709.04905.

[4] C. Finn and S. Levine. Deep visual foresight for planning robot motion. *CoRR*, abs/1610.00696, 2016. URL http://arxiv.org/abs/1610.00696.

[5] C. Finn, X. Y. Tan, Y. Duan, T. Darrell, S. Levine, and P. Abbeel. Learning visual feature spaces for robotic manipulation with deep spatial autoencoders. *CoRR*, abs/1509.06113, 2015. URL http://arxiv.org/abs/1509.06113.

[6] C. Devin, P. Abbeel, T. Darrell, and S. Levine. Deep object-centric representations for generalizable robot learning. *CoRR*, abs/1708.04225, 2017. URL http://arxiv.org/abs/1708.04225.

[7] S. Levine, C. Finn, T. Darrell, and P. Abbeel. End-to-end training of deep visuomotor policies. *CoRR*, abs/1504.00702, 2015. URL http://arxiv.org/abs/1504.00702.

[8] L. Yen-Chen, A. Zeng, S. Song, P. Isola, and T.-Y. Lin. Learning to see before learning to act: Visual pre-training for manipulation. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2020. URL https://yenchenlin.me/vision2action/.

[9] O. Kroemer, S. Niekum, and G. Konidaris. A review of robot learning for manipulation: Challenges, representations, and algorithms, 2019.

[10] K. M. Lynch and F. C. Park. Modern robotics: Mechanics, planning, and control. *Cambridge University Press*, 2017.

[11] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. M. O. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. Continuous control with deep reinforcement learning. *CoRR*, abs/1509.02971, 2016.

[12] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, P. Abbeel, and W. Zaremba. Hindsight experience replay. *CoRR*, abs/1707.01495, 2017. URL http://arxiv.org/abs/1707.01495.

[13] S. Feng, X. Xinjilefu, C. G. Atkeson, and J. Kim. Optimization based controller design and implementation for the atlas robot in the darpa robotics challenge finals. In *2015 IEEE-RAS 15th International Conference on Humanoid Robots (Humanoids)*, pages 1028–1035, 2015.

[14] OpenAI, M. Andrychowicz, B. Baker, M. Chociej, R. Józefowicz, B. McGrew, J. W. Pachocki, J. Pachocki, A. Petron, M. Plappert, G. Powell, A. Ray, J. Schneider, S. Sidor, J. Tobin, P. Welinder, L. Weng, and W. Zaremba. Learning dexterous in-hand manipulation. *CoRR*, abs/1808.00177, 2018. URL http://arxiv.org/abs/1808.00177.

[15] G. Neumann, C. Daniel, A. Paraschos, A. Kupcsik, and J. Peters. Learning modular policies for robotics. *Frontiers in Computational Neuroscience*, 8:62, 2014. ISSN 1662-5188. doi:10.3389/fncom.2014.00062. URL https://www.frontiersin.org/article/10.3389/fncom.2014.00062.

[16] B. D. Silva, G. Konidaris, and A. Barto. Learning parameterized skills, 2012.

[17] S. Schaal. *Dynamic Movement Primitives -A Framework for Motor Control in Humans and Humanoid Robotics*, pages 261–280. Springer Tokyo, Tokyo, 2006. ISBN 978-4-431-31381-6. doi:10.1007/4-431-31381-8_23. URL https://doi.org/10.1007/4-431-31381-8_23.

[18] E. Rückert and A. D'Avella. Learned parametrized dynamic movement primitives with shared synergies for controlling robotic and musculoskeletal systems. *Frontiers in Computational Neuroscience*, 7:138, 2013. ISSN 1662-5188. doi:10.3389/fncom.2013.00138. URL https://www.frontiersin.org/article/10.3389/fncom.2013.00138.

[19] F. Stulp, E. Theodorou, and S. Schaal. Reinforcement learning with sequences of motion primitives for robust manipulation. *Robotics, IEEE Transactions on*, 28:1360–1370, 12 2012. doi:10.1109/TRO.2012.2210294.

[20] O. Kroemer and G. S. Sukhatme. Learning relevant features for manipulation skills using meta-level priors. *CoRR*, abs/1605.04439, 2016. URL http://arxiv.org/abs/1605.04439.

[21] R. Strudel, A. Pashevich, I. Kalevatykh, I. Laptev, J. Sivic, and C. Schmid. Learning to combine primitive skills: A step towards versatile robotic manipulation, 2019.

[22] N. Shrivatsav, L. Nair, and S. Chernova. Tool substitution with shape and material reasoning using dual neural networks, 2019.

[23] O. Kilinc, Y. Hu, and G. Montana. Reinforcement learning for robotic manipulation using simulated locomotion demonstrations, 2019.

[24] N. Wahlström, T. B. Schön, and M. P. Deisenroth. From pixels to torques: Policy learning with deep dynamical models. *CoRR*, abs/1502.02251, 2015. URL http://arxiv.org/abs/

1502.02251.

[25] N. Ratliff, J. A. Bagnell, and S. S. Srinivasa. Imitation learning for locomotion and manipulation. In *2007 7th IEEE-RAS International Conference on Humanoid Robots*, pages 392–397, 2007.

[26] T. Zhang, Z. McCarthy, O. Jow, D. Lee, K. Goldberg, and P. Abbeel. Deep imitation learning for complex manipulation tasks from virtual reality teleoperation. *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1–8, 2018.

[27] M. Sieb, X. Zhou, A. Huang, O. Kroemer, and K. Fragkiadaki. Graph-structured visual imitation. In *Conference on Robot Learning (CoRL)*, 2019.

[28] A. Rajeswaran, V. Kumar, A. Gupta, J. Schulman, E. Todorov, and S. Levine. Learning complex dexterous manipulation with deep reinforcement learning and demonstrations. *CoRR*, abs/1709.10087, 2017. URL http://arxiv.org/abs/1709.10087.

[29] L. Fan, Y. Zhu, J. Zhu, Z. Liu, O. Zeng, A. Gupta, J. Creus-Costa, S. Savarese, and L. Fei-Fei. Surreal: Open-source reinforcement learning framework and robot manipulation benchmark. In A. Billard, A. Dragan, J. Peters, and J. Morimoto, editors, *Proceedings of The 2nd Conference on Robot Learning*, volume 87 of *Proceedings of Machine Learning Research*, pages 767–782. PMLR, 29–31 Oct 2018. URL http://proceedings.mlr.press/v87/fan18a.html.

[30] OpenAI, I. Akkaya, M. Andrychowicz, M. Chociej, M. Litwin, B. McGrew, A. Petron, A. Paino, M. Plappert, G. Powell, R. Ribas, J. Schneider, N. Tezak, J. Tworek, P. Welinder, L. Weng, Q. Yuan, W. Zaremba, and L. Zhang. Solving rubik's cube with a robot hand, 2019.

[31] M. A. Riedmiller, R. Hafner, T. Lampe, M. Neunert, J. Degrave, T. V. de Wiele, V. Mnih, N. Heess, and J. T. Springenberg. Learning by playing - solving sparse reward tasks from scratch. *CoRR*, abs/1802.10567, 2018. URL http://arxiv.org/abs/1802.10567.

[32] L. Pinto, M. Andrychowicz, P. Welinder, W. Zaremba, and P. Abbeel. Asymmetric actor critic for image-based robot learning. *CoRR*, abs/1710.06542, 2017. URL http://arxiv.org/abs/1710.06542.

[33] T. Tosun, E. Mitchell, B. Eisner, J. Huh, B. Lee, D. Lee, V. Isler, H. S. Seung, and D. D. Lee. Pixels to plans: Learning non-prehensile manipulation by imitating a planner. *CoRR*, abs/1904.03260, 2019. URL http://arxiv.org/abs/1904.03260.

[34] K. Kansky, T. Silver, D. A. Mély, M. Eldawy, M. Lázaro-Gredilla, X. Lou, N. Dorfman, S. Sidor, D. S. Phoenix, and D. George. Schema networks: Zero-shot transfer with a generative causal model of intuitive physics. *CoRR*, abs/1706.04317, 2017. URL http://arxiv.org/abs/1706.04317.

[35] V. F. Zambaldi, D. Raposo, A. Santoro, V. Bapst, Y. Li, I. Babuschkin, K. Tuyls, D. P. Reichert, T. P. Lillicrap, E. Lockhart, M. Shanahan, V. Langston, R. Pascanu, M. Botvinick, O. Vinyals, and P. W. Battaglia. Relational deep reinforcement learning. *CoRR*, abs/1806.01830, 2018. URL http://arxiv.org/abs/1806.01830.

[36] P. Florence, L. Manuelli, and R. Tedrake. Self-supervised correspondence in visuomotor policy learning, 2019.

[37] Z. Qin, K. Fang, Y. Zhu, L. Fei-Fei, and S. Savarese. Keto: Learning keypoint representations for tool manipulation. *arXiv preprint arXiv:1910.11977*, 2019.

[38] H.-Y. F. Tung, R. Cheng, and K. Fragkiadaki. Learning spatial common sense with geometry-aware recurrent networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019.

[39] J. J. Gibson. *The Ecological Approach to Visual Perception*. Houghton Mifflin, 1979.

[40] E. Todorov, T. Erez, and Y. Tassa. Mujoco: A physics engine for model-based control. In *IROS*, pages 5026–5033. IEEE, 2012. ISBN 978-1-4673-1737-5. URL http://dblp.uni-trier.de/db/conf/iros/iros2012.html#TodorovET12.

[41] A. X. Chang, T. Funkhouser, L. Guibas, P. Hanrahan, Q. Huang, Z. Li, S. Savarese, M. Savva, S. Song, H. Su, J. Xiao, L. Yi, and F. Yu. ShapeNet: An Information-Rich 3D Model Repository. Technical Report arXiv:1512.03012 [cs.GR], Stanford University — Princeton University — Toyota Technological Institute at Chicago, 2015.

[42] S. Ross, G. J. Gordon, and J. A. Bagnell. No-regret reductions for imitation learning and structured prediction. *CoRR*, abs/1011.0686, 2010. URL http://arxiv.org/abs/1011.0686.

[43] A. Mousavian, C. Eppner, and D. Fox. 6-dof graspnet: Variational grasp generation for object manipulation. *CoRR*, abs/1905.10520, 2019. URL http://arxiv.org/abs/1905.10520.

[44] A. W. Harley, F. Li, S. K. Lakshmikanth, X. Zhou, H. F. Tung, and K. Fragkiadaki. Embodied view-contrastive 3d feature learning. *CoRR*, abs/1906.03764, 2019. URL http://arxiv.org/abs/1906.03764.